Title: Optimization of Multilevel Feedback Queue Scheduling and Queueing Models Using Simulation and Analytical Techniques

Sidrit RRUSTEMI, Viola ZENELI, Artemisa HASALAMI

Department of Computer Engineering, EPOKA University, Tirana, Albania

Abstract

CPU scheduling is one of the most important functions of an operating system because it directly affects system responsiveness, resource utilization, and overall performance. Among the various scheduling algorithms, the Multilevel Feedback Queue (MLFQ) scheduler is widely used due to its ability to dynamically adjust process priorities based on execution behavior. However, the performance of MLFQ strongly depends on the selection of scheduling parameters such as queue quantum values and allocation policies.

This paper investigates the optimization of MLFQ scheduling using simulation and analytical techniques. A Python-based simulation framework was developed to generate stochastic workloads with random burst times and arrival patterns. The scheduler was evaluated using performance metrics including throughput, turnaround time, waiting time, and response time. Grid-search optimization combined with statistical averaging over 100 simulation runs was applied to identify efficient parameter configurations.

To complement the simulation study, queueing theory models were incorporated using M/M/1 and M/M/S systems. These models were used to analyze utilization, queue length, waiting time, and system stability under different workload conditions. In addition, a randomness study was conducted to compare Uniform, Exponential, and Poisson workload distributions and evaluate their impact on scheduling performance.

The results demonstrate that scheduler parameter selection has a significant influence on performance. Smaller queue quantum values generally reduced waiting time and improved responsiveness, while larger quantum values increased process delays. The queueing analysis confirmed the importance of maintaining stable utilization levels, and the randomness study showed that workload characteristics affect waiting and response times. Overall, the study highlights the effectiveness of combining simulation, optimization, and analytical modeling to improve CPU scheduling performance.

Keywords: CPU Scheduling, Multilevel Feedback Queue (MLFQ), Queueing Theory, Scheduling Optimization, Grid Search, Monte Carlo Simulation

1. Introduction

1.1 Background and Motivation

CPU scheduling is a fundamental function of modern operating systems, responsible for determining how processor resources are allocated among competing processes. The effectiveness of a scheduling algorithm directly affects system performance, resource utilization, and user experience. Common performance indicators include throughput, turnaround time, waiting time, and response time.

Among the various scheduling algorithms, the Multilevel Feedback Queue (MLFQ) scheduler is widely used because it dynamically adjusts process priorities according to their execution behavior. Interactive and short-running processes are typically favored through higher-priority queues, while CPU-intensive processes are gradually moved to lower-priority levels. This adaptive behavior allows MLFQ to balance responsiveness and fairness more effectively than many traditional scheduling approaches.

However, the performance of MLFQ depends heavily on several configurable parameters, including queue time quantum values, queue allocation limits, and scheduling thresholds. Different parameter selections can significantly influence scheduling efficiency, making it difficult to identify a configuration that performs well under all workload conditions. Furthermore, real-world systems are characterized by stochastic process arrivals and varying execution times, requiring systematic evaluation methods to understand scheduler behavior.

To address these challenges, this study investigates the optimization of MLFQ scheduling through simulation and analytical modeling. A configurable simulation framework is used to evaluate multiple parameter combinations under randomly generated workloads, while queueing theory models are employed to analyze system stability, utilization, and queue behavior.

1.2 Research Objectives

The objectives of this research are:

- To develop a simulation model of the Multilevel Feedback Queue scheduler.
- To evaluate scheduling performance using throughput, turnaround time, waiting time, and response time metrics.
- To determine optimal scheduling parameters through grid-search optimization and statistical averaging.
- To analyze queueing behavior using M/M/1 and M/M/S models.
- To compare the effects of uniform, exponential, and Poisson workload distributions on system performance.

1.3 Paper Organization

The remainder of this paper is organized as follows. Section 2 presents the system model and workload generation process. Section 3 describes the MLFQ simulation model and scheduling parameters. Section 4 introduces the performance metrics used in the evaluation. Section 5 discusses the simulation framework and experimentation methodology. Section 6 presents the optimization study, while Section 7 introduces the visualization techniques used for analysis. Section 8 extends the study through queueing theory models, and Section 9 investigates the impact of randomness on scheduling performance. Section 10 presents the experimental results, and Section 11 concludes the paper.

2. System Model

2.1 Process Generation

The simulation models a single-processor operating system in which multiple processes compete for CPU resources. Each process is characterized by an arrival time and a CPU burst time.

To represent diverse workloads, burst times are generated randomly within the range [10,1000].

Process arrivals are stochastic and follow a selected probability distribution. Different arrival models, including uniform, exponential, and Poisson distributions, are later analyzed to evaluate their impact on scheduling performance.

2.2 Scheduling Environment

A Multilevel Feedback Queue (MLFQ) scheduler is used to manage process execution. The scheduler contains multiple priority levels and dynamically adjusts process priorities according to their execution behavior. New processes are initially assigned to the highest-priority queue and may move between queues during execution.

The simulation assumes a single CPU, independent processes, and no I/O blocking. Context-switch overhead is ignored, and all generated processes are assumed to complete execution. These assumptions allow the study to focus on the impact of scheduling parameters without introducing additional system-level complexities.

2.3 Simulation Parameters

The behavior of the scheduler is controlled through several configurable parameters:

1. Q1 – Time quantum of the first queue.
2. Q2 – Time quantum of the second queue.
3. L1 – CPU allocation level in Q1.
4. L2 – CPU allocation level in Q2.
5. T – Threshold used in the scheduling policy of the final queue.

Different parameter combinations are evaluated throughout the study to determine their effect on system performance.

3. MLFQ Simulation

3.1 Simulation Model

The Multilevel Feedback Queue scheduler serves as the core component of the simulation framework. Its objective is to provide a balance between responsiveness and efficiency by dynamically adjusting process priorities based on CPU usage. Short processes are expected to complete in higher-priority queues, while longer CPU-bound processes gradually move to lower-priority levels.

Each simulation run begins with a randomly generated workload. Processes enter the scheduler according to their arrival times and remain in the system until their execution is completed.

3.2 Scheduling Mechanism

When a process arrives, it is inserted into the highest-priority queue. The scheduler allocates CPU time according to the time quantum assigned to the current queue. If the process completes execution, it exits the system. Otherwise, its remaining burst time is updated and the process may be transferred to a lower-priority queue.

This feedback-based mechanism enables short jobs to receive rapid service while ensuring that long-running processes continue to make progress. As a result, the scheduler attempts to balance response time and throughput across different workload conditions.

3.3 Parameter Configuration

The simulation investigates the effect of several MLFQ parameters:

1. Q1: Time quantum assigned to the first queue.
2. Q2: Time quantum assigned to the second queue.
3. L1: Maximum CPU allocation before demotion from the first queue.
4. L2: Maximum CPU allocation before demotion from the second queue.
5. T: Threshold used to determine whether the lowest-priority queue applies FCFS or SJF scheduling.

The performance of the scheduler is expected to vary depending on these parameter values. Smaller quantum values generally improve responsiveness but may increase context switching, whereas larger quantum values can improve throughput at the cost of slower response times.

3.4 Experimental Configurations

Multiple parameter combinations are evaluated during the simulation study. Example values include:

Parameter	Tested Values
Q1	10, 20, 30, 40
Q2	40, 60, 80, 100
L1	1, 2, 3
L2	2, 3, 4
T	100, 200, 300

The results obtained from these configurations are analyzed using the performance metrics presented in the next section. Statistical averages from multiple simulation runs are used to identify parameter combinations that provide the most efficient scheduling behavior.

4. Performance Metrics

The performance of the proposed Multilevel Feedback Queue scheduler is evaluated using four commonly used scheduling metrics: throughput, turnaround time, waiting time, and response time. These metrics provide a comprehensive view of scheduler efficiency and allow meaningful comparisons between different parameter configurations.

4.1 Throughput

Throughput represents the number of processes completed during a given period of time. It measures the overall productivity of the scheduling algorithm and indicates how efficiently the processor is utilized.

$$Throughput = \frac{Number\ of\ Completed\ Processes}{Total\ Simulation\ Time}$$

Higher throughput values generally indicate better system performance because more processes are completed within the same time interval.

4.2 Turnaround Time

Turnaround time measures the total time required for a process to complete execution after entering the system. It includes waiting time as well as actual CPU execution time.

$$Turnaround\ Time = \frac{Completion\ Time}{Arrival\ Time}$$

A lower turnaround time indicates that processes are completed more quickly, improving overall system efficiency.

4.3 Waiting Time

Waiting time represents the amount of time a process spends in ready queues while waiting for CPU allocation. Excessive waiting times can reduce system responsiveness and negatively affect user experience.

$$Waiting\ Time = \frac{Turnaround\ Time}{Burst\ Time}$$

An effective scheduling algorithm should minimize waiting time while maintaining fairness among processes.

4.4 Response Time

Response time measures the delay between the arrival of a process and its first allocation of CPU time. This metric is particularly important for interactive systems where users expect immediate feedback.

$$Response\ Time\ = \frac{First\ Execution\ Time}{Arrival\ Time}$$

Lower response times indicate better responsiveness and improved user satisfaction.

4.5 Evaluation Criteria

The objective of the optimization study is to maximize throughput while minimizing turnaround time, waiting time, and response time. These metrics are collected during each simulation run and averaged across multiple experiments to evaluate the effectiveness of different MLFQ parameter configurations.

5. Simulation Framework

5.1 Implementation

To evaluate the performance of the Multilevel Feedback Queue (MLFQ) scheduler, a simulation framework was developed in Python. The simulator models process arrivals, CPU burst times, queue transitions, and scheduling decisions according to the MLFQ algorithm. During execution, the program records performance metrics such as throughput, turnaround time, waiting time, and response time.

The source code was developed specifically for this study and is provided in the appendix.

5.2 Workload Generation

For each experiment, processes were generated with random arrival times and burst times. CPU burst times were randomly selected within the range of 10 to 1000 time units in order to represent both short and long-running processes.

The generated workloads were then submitted to the scheduler for execution, allowing the performance of different parameter configurations to be evaluated under varying operating conditions.

5.3 Monte Carlo Simulation

A Monte Carlo simulation approach was used to improve the reliability of the results. Instead of evaluating a single workload, the scheduler was executed repeatedly using newly generated process sets.

For each parameter configuration, 100 independent simulation trials were performed. Performance metrics obtained from all trials were averaged to reduce the impact of random variations and provide more stable results.

The average value of each metric was calculated as:

$$\bar{X} = \frac{1}{n}\sum_{i=1}^{n} X_i$$

where X_i represents the result of a single simulation run and (n=100) is the total number of trials.

5.4 Data Collection

During each simulation run, the framework recorded the following information:

1. Throughput
2. Turnaround time
3. Waiting time
4. Response time
5. Number of completed processes

The collected data were stored and later used for statistical analysis and parameter optimization.

5.5 Experimental Procedure

The experiments were performed using the following procedure:

- Generate a random workload.
- Configure the MLFQ scheduler parameters.
- Execute all processes using the selected configuration.
- Record the resulting performance metrics.
- Repeat the simulation for 100 trials.
- Compute average performance values.
- Compare the results obtained from different parameter combinations.

This framework provides a consistent and repeatable method for evaluating scheduler performance and identifying parameter settings that improve overall system efficiency.

6. Optimization Study

6.1 Grid Search Methodology

The objective of the optimization study was to identify efficient parameter configurations for the Multilevel Feedback Queue (MLFQ) scheduler. Since scheduler performance depends heavily on the selection of queue quantum values and allocation policies, a grid-search approach was employed to systematically evaluate different parameter combinations.

The following parameters were varied during the experiments:

Table 6.1. *Evaluated MLFQ Parameters*

Parameter	Tested Values
Q1	10, 20, 30, 40
Q2	40, 60, 80, 100
L1	1, 2, 3
L2	2, 3, 4
T	100, 200, 300

For each parameter configuration, the simulation was executed 100 times using independently generated workloads. Statistical averaging was then applied to reduce the influence of random variations and obtain more reliable results. The optimization process evaluated scheduler performance using average waiting time, turnaround time, response time, and throughput.

6.2 Average Waiting Time Analysis

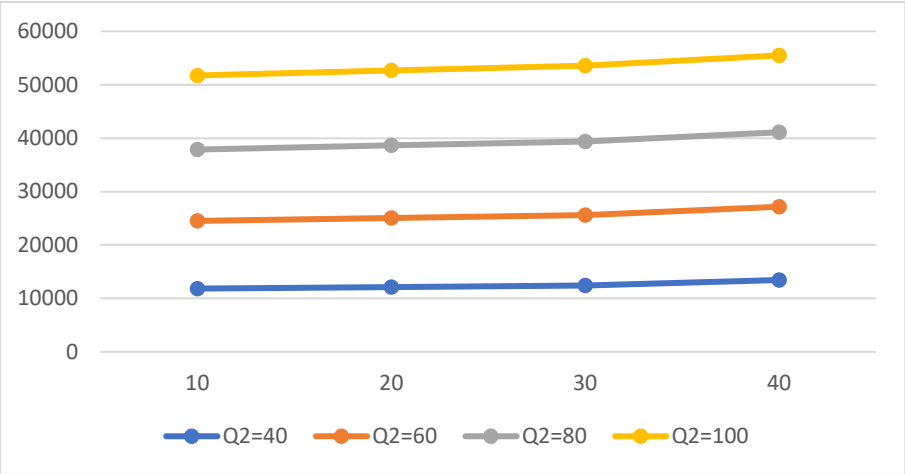
Among all performance metrics, average waiting time was selected as the primary optimization objective because it directly reflects scheduler efficiency from the perspective of process responsiveness.

The average waiting times obtained for different combinations of Q1 and Q2 are shown in Table 6.2 and Chart 6.1.

Table 6.2. *Average Waiting Time for Different Quantum Configurations*

Q1	Q2=40	Q2=60	Q2=80	Q2=100
10	11850.8	12665.9	13346.7	13875.7
20	12141.3	12918.4	13566.2	14066.9
30	12417.9	13163.9	13770.8	14235.5
40	13453.3	13708.7	13949.6	14380.5

Chart 6.1. *Average Waiting Time for Different Quantum Configurations Chart*



The results demonstrate a clear trend. As both Q1 and Q2 increase, the average waiting time also increases. Smaller quantum values allow the scheduler to switch between processes more frequently, reducing the time that processes spend waiting in ready queues. Conversely, larger quantum values cause processes to occupy the CPU for longer periods, increasing the waiting time experienced by other processes.

6.3 Best and Worst Configurations

The optimization process identified both the best-performing and worst-performing configurations within the tested parameter space.

Table 6.3. *Best and Worst Scheduling Configurations*

Configuration	Q1	Q2	Average Waiting Time
Best	10	40	11850.8
Worst	40	100	14380.5

The best-performing configuration achieved the lowest waiting time when Q1 and Q2 used the smallest tested quantum values. In contrast, the worst-performing configuration occurred when both quantum values were set to their maximum levels.

The difference between these configurations exceeds 2500 time units, indicating that parameter selection has a substantial influence on scheduler performance. This finding highlights the importance of optimization rather than relying on arbitrary scheduler settings.

6.4 Discussion

The optimization study confirms that MLFQ performance is highly sensitive to parameter configuration. Lower quantum values improve responsiveness because processes receive CPU attention more frequently. However, excessively large quantum values increase waiting time and reduce overall scheduling efficiency.

The results suggest that selecting appropriate scheduler parameters is essential for achieving balanced system performance. Grid-search optimization proved effective for identifying efficient configurations and provides a practical method for tuning scheduling algorithms in operating systems.

7. Visualization

7.1 Purpose of Visualization

While numerical tables provide detailed information about scheduler performance, graphical visualizations make it easier to identify patterns and trends. Therefore, heatmaps and three-dimensional performance surfaces were generated to visualize the relationship between MLFQ parameters and scheduling efficiency.

The visualizations focus primarily on average waiting time because it was the main optimization objective of the study.

7.2 Heatmap Analysis

Figure 7.1. Heatmap of Average Waiting Time

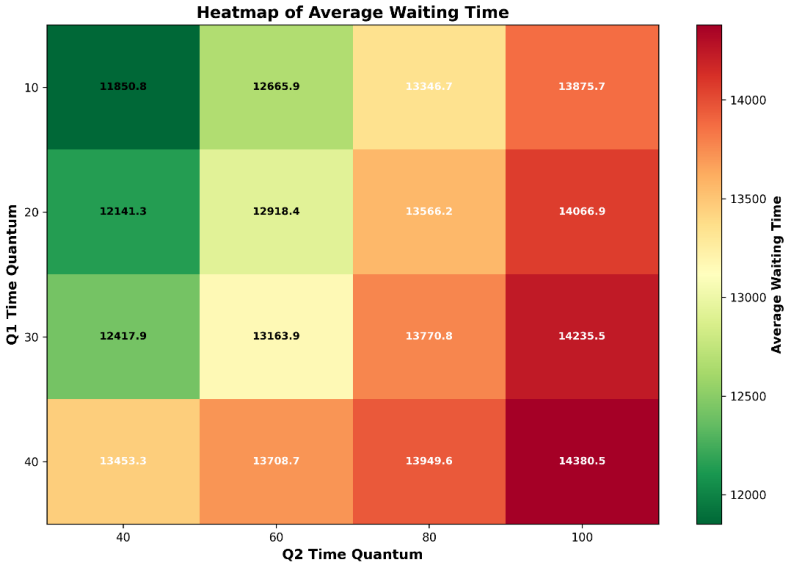


Figure 7.1. presents a heatmap of average waiting time for different combinations of Q1 and Q2.

The heatmap clearly illustrates the effect of increasing queue quantum values. The lowest waiting times are concentrated in the upper-left region of the figure, corresponding to Q1 = 10 and Q2 = 40. As the quantum values increase, the waiting time gradually rises, producing darker and more intense color regions.

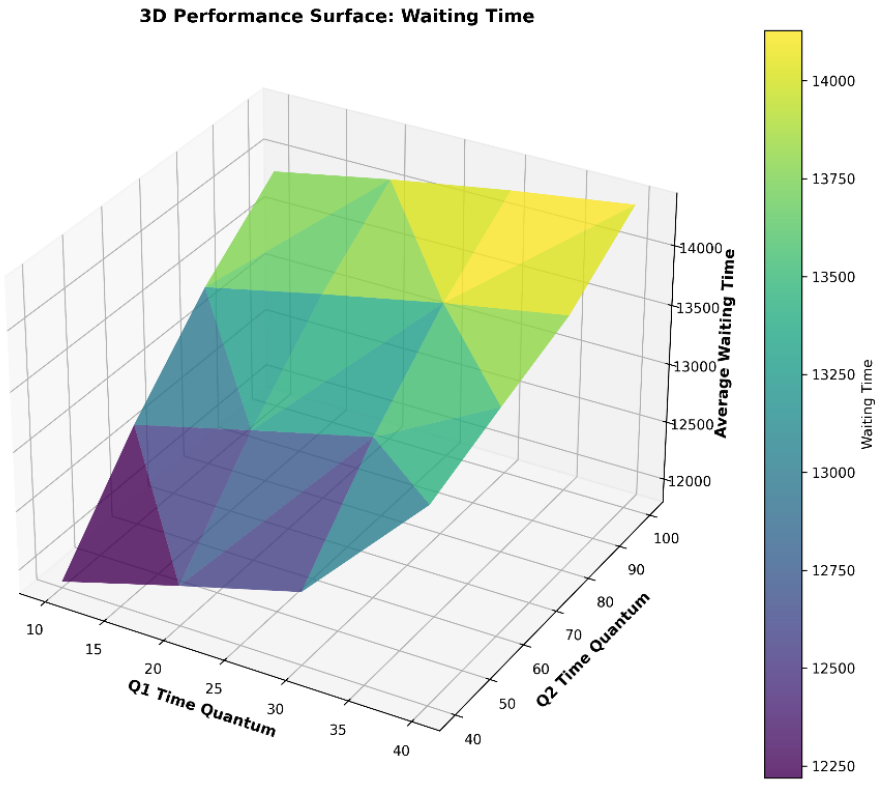
The minimum waiting time observed in the heatmap is approximately 11850.8 time units, while the maximum exceeds 14380 time units. This confirms the numerical findings presented in Section 6.

The visualization also demonstrates that Q2 has a particularly strong impact on waiting time. Increasing Q2 from 40 to 100 consistently increases waiting time regardless of the value selected for Q1.

7.3 Three-Dimensional Performance Surface

Although heatmaps provide a useful two-dimensional representation, they cannot fully illustrate how waiting time changes across the parameter space. To address this limitation, a three-dimensional performance surface was generated.

Figure 7.2. *Three-Dimensional Performance Surface for Waiting Time*



The three-dimensional surface reveals a clear upward trend as both Q1 and Q2 increase. Regions of low waiting time appear as valleys, while higher waiting times form elevated areas of the surface.

The lowest point on the surface corresponds to the optimal parameter configuration identified during the optimization study. Similarly, the highest point corresponds to the largest tested quantum values.

The smooth shape of the surface indicates a strong relationship between quantum size and scheduler performance. This relationship suggests that waiting time can be reduced significantly through appropriate parameter selection.

7.4 Interpretation of Visualization Results

Both visualizations support the conclusions obtained from the optimization study. The heatmap and performance surface consistently show that smaller quantum values produce lower waiting times and therefore improve scheduler responsiveness.

These visualizations also provide an intuitive representation of the optimization process, allowing efficient parameter regions to be identified quickly without examining individual numerical results.

Overall, the graphical analysis confirms that careful tuning of MLFQ parameters can significantly improve scheduling performance and reduce process waiting time.

8. Queueing Theory Extension

8.1 Overview

Queueing theory is widely used to analyze systems in which requests arrive, wait for service, and leave after being processed. In computer systems, processes waiting for CPU time can be modeled as customers in a queue, while the processor acts as the server. Queueing models provide useful analytical measures such as utilization, queue length, waiting time, and system stability.

In this study, queueing theory complements the MLFQ simulation by providing a mathematical perspective on system behavior under different workload conditions.

8.2 M/M/1 Model

The M/M/1 model describes a system with a single server. In the context of CPU scheduling, this represents a single processor handling arriving processes. The first "M" indicates that arrivals follow a Poisson process, while the second "M" indicates exponentially distributed service times.

The utilization of the system is calculated as:

$$\rho = \frac{\lambda}{\mu}$$

where λ is the arrival rate and μ is the service rate.

For stable operation, utilization must remain below one:

$$\rho < 1$$

When utilization approaches one, processes arrive almost as quickly as they can be served, resulting in increased waiting times and longer queues.

The average queue length is given by:

$$L_q = \frac{\rho^2}{1 - \rho}$$

while the average waiting time is:

$$W_q = \frac{\rho}{\mu - \lambda}$$

These equations demonstrate that system performance deteriorates rapidly as the arrival rate approaches the service rate.

8.3 M/M/S Model

The M/M/S model extends the single-server model by introducing multiple servers. In operating systems, this can represent multi-core processors where several processes may execute simultaneously.

The utilization of the system is calculated as:

$$\rho = \frac{\lambda}{S\mu}$$

where *S* is the number of servers.

Because the workload is distributed among multiple servers, the M/M/S model generally achieves lower waiting times and improved throughput compared to the M/M/1 model. Increasing the number of servers also improves system stability by reducing processor congestion.

8.4 Stability Analysis

System stability is an important aspect of queueing theory. A stable system is one in which processes are serviced faster than they arrive on average. If arrivals occur too frequently, the queue continues to grow and waiting times become increasingly large.

For the M/M/1 model, stability requires:

$$\lambda < \mu$$

For the M/M/S model, the condition becomes:

$$\lambda < S\mu$$

These conditions show that adding additional servers increases the service capacity of the system and allows it to handle higher arrival rates without becoming overloaded.

Table 8.1. *Queueing Model Results*

Model	λ	μ	Servers (S)	Utilization (ρ)	Stable
M/M/1	5	10	1	0.50	Yes
M/M/1	8	10	1	0.80	Yes
M/M/1	12	10	1	1.20	No
M/M/S	12	10	2	0.60	Yes
M/M/S	20	10	4	0.50	Yes
M/M/S	30	10	4	0.75	Yes

Table 8.1 illustrates the effect of increasing arrival rates and server counts on system utilization. The single-server M/M/1 model becomes unstable when the arrival rate exceeds the service rate, resulting in utilization greater than one. In contrast, the M/M/S configurations remain stable because the workload is distributed across multiple servers, reducing congestion and improving overall system performance.

Figure 8.1. *Queueing Model Utilization Comparison*

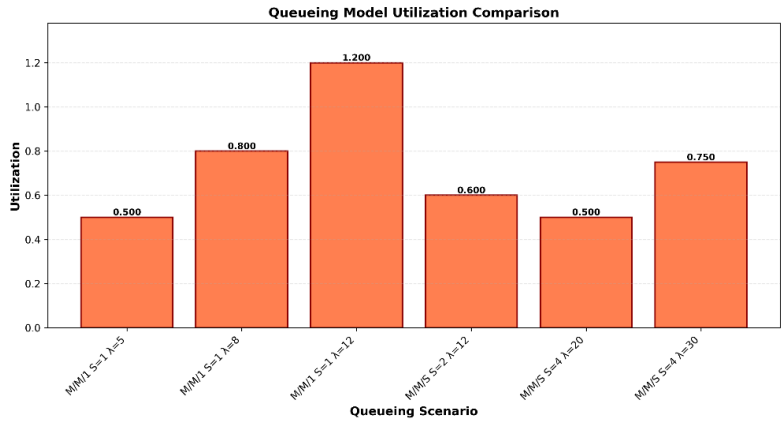


Figure 8.1 visualizes the utilization values obtained from the queueing analysis. The unstable M/M/1 configuration reaches a utilization of 1.20, indicating that processes arrive faster than they can be served. The multi-server configurations maintain lower utilization values, demonstrating the benefits of additional processing capacity. The results confirm that increasing the number of servers improves stability and reduces the risk of excessive queue growth.

8.5 Relationship to Scheduling Performance

The queueing models support the simulation study by explaining the effects of workload intensity on scheduler performance. As utilization increases, processes spend more time waiting for CPU access, leading to higher turnaround and response times. Similarly, lower utilization levels result in shorter queues and improved responsiveness.

Although queueing theory does not model the detailed behavior of the MLFQ scheduler, it provides useful insight into the general relationship between workload demand and system performance. Therefore, the analytical results obtained from M/M/1 and M/M/S models complement the simulation results presented later in this paper.

9. Randomness Study

9.1 Overview

The performance of CPU scheduling algorithms is influenced not only by scheduler parameters but also by the characteristics of the workload. Different process arrival patterns can create significantly different scheduling conditions, even when the same scheduling algorithm is used.

To investigate the impact of workload randomness, three commonly used arrival distributions were evaluated:

1. Uniform distribution
2. Exponential distribution
3. Poisson distribution

For each distribution, the simulation was executed 100 times using the same MLFQ configuration. The average waiting time, response time, and throughput were recorded and compared.

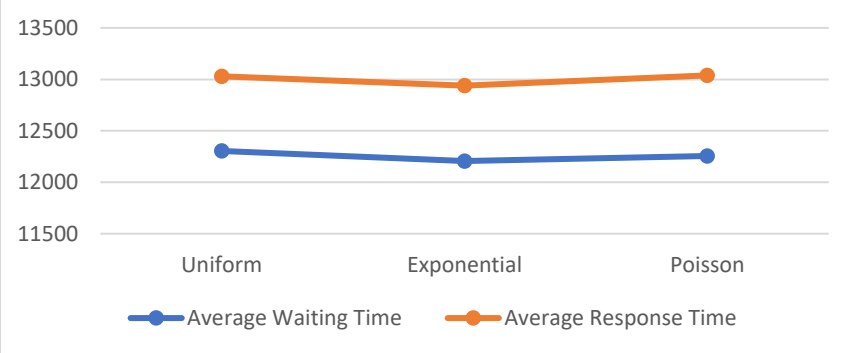
9.2 Experimental Results

The results obtained from the randomness study are presented in Table 9.1 and Chart 9.1.

Table 9.1. *Comparison of Workload Distributions*

Distribution	Average Waiting Time	Average Response Time	Throughput
Uniform	12304.04	726.37	0.0020
Exponential	12206.40	732.97	0.0020
Poisson	12256.39	782.14	0.0020

Table 9.1. *Comparison of Workload Distributions Chart*

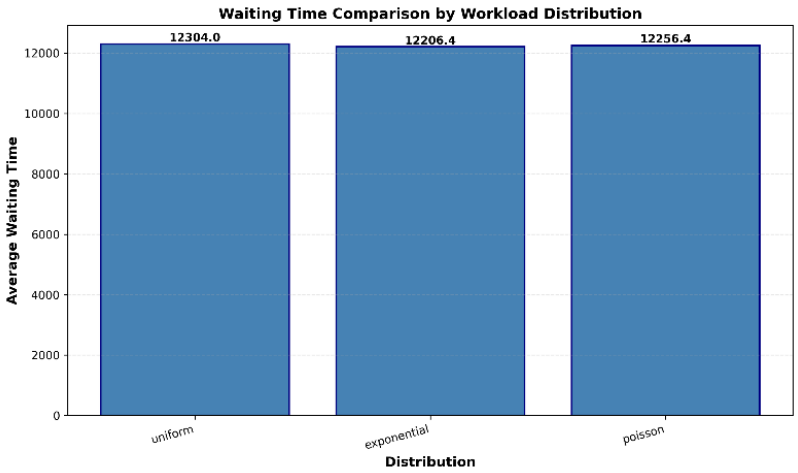


The results indicate that the exponential distribution produced the lowest average waiting time, while the Poisson distribution generated the highest average response time. Throughput remained almost identical across all workload distributions.

9.3 Distribution Comparison

To visualize the impact of workload randomness on scheduling performance, a comparison graph was generated using the average waiting time values obtained from the experiments.

Figure 9.1. *Waiting Time Comparison by Workload Distribution*



The figure shows that the differences between the three distributions are relatively small. However, measurable variations can still be observed. The exponential distribution produced the most favorable waiting-time results, while the uniform distribution generated slightly higher waiting times. The Poisson distribution produced values between the other two distributions but resulted in the highest response time.

9.4 Discussion

The experimental results demonstrate that workload distribution affects scheduler behavior even when the scheduling algorithm and parameter configuration remain unchanged.

The exponential distribution generated the lowest waiting time because process arrivals were more naturally dispersed throughout the simulation period, reducing contention for CPU resources. The Poisson distribution introduced greater variability in arrival patterns, which increased response times and created short periods of higher queue occupancy.

Despite these differences, throughput remained stable for all tested distributions. This suggests that the MLFQ scheduler maintained a consistent overall processing capacity regardless of workload randomness.

Overall, the randomness study shows that workload characteristics should be considered when evaluating scheduling algorithms. Although parameter optimization remains the primary factor affecting performance, process arrival distributions can influence waiting time and response time and therefore contribute to overall scheduling efficiency.

10. Results

10.1 Optimization Results

The optimization study demonstrated that MLFQ scheduler performance is highly dependent on the selection of queue quantum values. Through grid-search optimization, multiple parameter configurations were evaluated and compared using average waiting time as the primary performance metric.

The results showed that smaller quantum values consistently produced lower waiting times. The best-performing configuration was obtained with Q1 = 10 and Q2 = 40, achieving an average waiting time of 11850.8 time units. In contrast, the largest waiting time of 14380.5 time units was observed when Q1 = 40 and Q2 = 100.

These findings indicate that shorter time quanta improve scheduler responsiveness by allowing more frequent CPU allocation among waiting processes.

Table 10.1. *Best and Worst Configurations*

Configuration	Q1	Q2	Average Waiting Time
Best	10	40	11850.8
Worst	40	100	14380.5

The difference between the best and worst configurations exceeded 2500 time units, demonstrating the importance of parameter tuning in MLFQ scheduling.

10.2 Queueing Theory Results

The queueing theory analysis provided a mathematical interpretation of system behavior under different workload conditions. The M/M/1 and M/M/S models were evaluated using various arrival rates and service capacities.

The results showed that system stability strongly depends on utilization. For the M/M/1 model, stability was maintained only when the arrival rate remained below the service rate. When λ exceeded μ, utilization became greater than one and the system entered an unstable state.

The M/M/S model demonstrated improved stability because the workload was distributed among multiple servers. As the number of servers increased, utilization decreased and queue growth was reduced.

Table 10.2. *Summary of Queueing Results*

Model	Utilization Range	Stability
M/M/1	0.50 – 1.20	Stable only when $\rho < 1$
M/M/S	0.50 – 0.75	Stable in all tested cases

These findings confirm that additional processing capacity can significantly improve system performance and reduce congestion.

10.3 Randomness Study Results

The randomness study compared three workload distributions: Uniform, Exponential, and Poisson. The objective was to determine whether workload generation patterns influence scheduler performance.

Table 10.3. *Distribution Comparison*

Distribution	Average Waiting Time	Average Response Time
Uniform	12304.04	726.37
Exponential	12206.40	732.97
Poisson	12256.39	782.14

The results indicate that the exponential distribution produced the lowest waiting time, while the Poisson distribution generated the highest response time. Although throughput remained almost identical across all experiments, measurable differences were observed in waiting and response times.

These results suggest that workload characteristics influence scheduler performance and should be considered when evaluating CPU scheduling algorithms.

10.4 Overall Findings

The combined simulation and analytical studies produced several important observations:

- MLFQ performance is highly sensitive to scheduler parameter selection.
- Smaller queue quantum values generally reduce waiting time and improve responsiveness.
- Grid-search optimization successfully identified efficient parameter configurations.
- Queueing theory confirmed the importance of maintaining utilization below one to ensure system stability.
- Multi-server systems provide better stability and lower congestion than single-server systems.
- Workload distribution influences waiting time and response time, although throughput remains relatively stable.

Overall, the results demonstrate that scheduling performance can be improved significantly through careful parameter optimization and workload-aware analysis.

11. Conclusion

This paper investigated the optimization of Multilevel Feedback Queue (MLFQ) scheduling using simulation and analytical techniques. A Python-based simulation framework was developed to evaluate scheduler performance under randomly generated workloads with varying process arrival patterns and burst times.

The study focused on the impact of queue quantum values, queue allocation levels, and scheduling thresholds on system performance. Through grid-search optimization and statistical averaging over 100 simulation runs, multiple parameter configurations were evaluated using throughput, turnaround time, waiting time, and response time as performance metrics.

The experimental results demonstrated that scheduler parameters have a significant influence on overall performance. Smaller queue quantum values generally reduced waiting time and improved responsiveness, while larger quantum values increased process waiting times. The optimization process successfully identified parameter configurations that produced better scheduling efficiency.

To complement the simulation study, queueing theory models were applied to analyze utilization, queue length, waiting time, and system stability. The M/M/1 and M/M/S models provided a theoretical explanation of the observed behavior and confirmed that stable operation requires the service rate to exceed the arrival rate. The analysis also showed that multi-server systems offer improved stability and lower congestion compared to single-server systems.

A randomness study was additionally conducted using Uniform, Exponential, and Poisson workload distributions. Although throughput remained relatively stable across all distributions, differences were observed in waiting time and response time. These results demonstrate that workload characteristics can influence scheduling behavior and should be considered during scheduler evaluation.

Overall, the results of this study show that careful tuning of MLFQ parameters can significantly improve scheduling performance. The combination of simulation, optimization techniques, visualization, and queueing theory provides a comprehensive framework for analyzing CPU scheduling algorithms and understanding their behavior under different workload conditions.

Future work may extend this study by investigating additional scheduling algorithms, larger workload sizes, adaptive parameter tuning techniques, and more advanced queueing models to further improve operating system scheduling performance.

Appendix A

Code 1. *MLFQ Simulation Source Code*

```
import random
from dataclasses import dataclass
from typing import List, Dict


@dataclass
class Process:
    pid: int
    arrival_time: int
    burst_time: int
    remaining_time: int
    first_execution_time: int = -1
    completion_time: int = 0


def generate_processes(
    num_processes: int = 50,
    burst_min: int = 10,
    burst_max: int = 1000,
    arrival_distribution: str = "uniform",
    seed: int | None = None
) -> List[Process]:
    """
    Generates a random workload.
    Burst times are between 10 and 1000 CPU time units.
    Arrival times are stochastic.
    """
    if seed is not None:
        random.seed(seed)

    processes = []
    current_arrival = 0

    for pid in range(1, num_processes + 1):
        burst = random.randint(burst_min, burst_max)

        if arrival_distribution == "uniform":
            interarrival = random.randint(0, 20)
        elif arrival_distribution == "exponential":
            interarrival = int(random.expovariate(1 / 10))
        elif arrival_distribution == "poisson":
            # Approximation of Poisson-style arrival gaps using exponential interarrival time
            interarrival = int(random.expovariate(1 / 8))
        else:
            raise ValueError("Invalid arrival distribution. Use uniform, exponential, or poisson.")

        current_arrival += interarrival

        processes.append(Process(
            pid=pid,
```

```
            arrival_time=current_arrival,
            burst_time=burst,
            remaining_time=burst
        ))

    return processes


def run_mlfq(
    processes: List[Process],
    q1: int,
    q2: int,
    l1: int,
    l2: int,
    threshold_t: int
) -> Dict[str, float]:
    """
    Runs a simplified three-level MLFQ scheduler:
    Q1: Round Robin
    Q2: Round Robin
    Q3: FCFS or SJF depending on threshold T
    """
    processes = sorted(processes, key=lambda p: p.arrival_time)
    time = 0
    completed = []
    not_arrived = processes[:]

    queue1 = []
    queue2 = []
    queue3 = []

    def add_arrivals():
        nonlocal not_arrived
        arrived = [p for p in not_arrived if p.arrival_time <= time]
        queue1.extend(arrived)
        not_arrived = [p for p in not_arrived if p.arrival_time > time]

    while len(completed) < len(processes):
        add_arrivals()

        if not queue1 and not queue2 and not queue3:
            if not_arrived:
                time = not_arrived[0].arrival_time
                add_arrivals()
            else:
                break

        if queue1:
            process = queue1.pop(0)
            quantum = q1 * l1
        elif queue2:
            process = queue2.pop(0)
            quantum = q2 * l2
        else:
            # T decides whether Q3 behaves more like SJF or FCFS.
            if threshold_t <= 200:
```

```
            queue3.sort(key=lambda p: p.remaining_time)
        process = queue3.pop(0)
        quantum = process.remaining_time

    if process.first_execution_time == -1:
        process.first_execution_time = time

    execution_time = min(quantum, process.remaining_time)
    process.remaining_time -= execution_time
    time += execution_time

    add_arrivals()

    if process.remaining_time == 0:
        process.completion_time = time
        completed.append(process)
    else:
        if quantum == q1 * l1:
            queue2.append(process)
        elif quantum == q2 * l2:
            queue3.append(process)
        else:
            queue3.append(process)

total_time = max(p.completion_time for p in completed) if completed else 1

turnaround_times = [p.completion_time - p.arrival_time for p in completed]
waiting_times = [tat - p.burst_time for tat, p in zip(turnaround_times, completed)]
response_times = [p.first_execution_time - p.arrival_time for p in completed]

return {
    "throughput": len(completed) / total_time,
    "avg_turnaround_time": sum(turnaround_times) / len(turnaround_times),
    "avg_waiting_time": sum(waiting_times) / len(waiting_times),
    "avg_response_time": sum(response_times) / len(response_times),
    "completed_processes": len(completed),
    "total_time": total_time
}


if __name__ == "__main__":
    workload = generate_processes(num_processes=50, arrival_distribution="uniform", seed=1)

    results = run_mlfq(
        processes=workload,
        q1=20,
        q2=60,
        l1=2,
        l2=3,
        threshold_t=200
    )

    print("MLFQ Simulation Results")
    print("----------------------")
    for key, value in results.items():
        print(f"{key}: {value:.4f}" if isinstance(value, float) else f"{key}: {value}")
```

Code 2. *Grid Search Optimization Source Code*

```
import csv
from mlfq_simulation import generate_processes, run_mlfq


Q1_VALUES = [10, 20, 30, 40]
Q2_VALUES = [40, 60, 80, 100]
L1_VALUES = [1, 2, 3]
L2_VALUES = [2, 3, 4]
T_VALUES = [100, 200, 300]

TRIALS = 100
NUM_PROCESSES = 50


def average_results(results_list):
    keys = results_list[0].keys()
    return {
        key: sum(r[key] for r in results_list) / len(results_list)
        for key in keys
    }


def main():
    output_file = "optimization_results.csv"
    rows = []

    print("Starting grid search optimization...")
    print(f"Trials per configuration: {TRIALS}")

    for q1 in Q1_VALUES:
        for q2 in Q2_VALUES:
            for l1 in L1_VALUES:
                for l2 in L2_VALUES:
                    for t in T_VALUES:
                        trial_results = []

                        for trial in range(TRIALS):
                            workload = generate_processes(
                                num_processes=NUM_PROCESSES,
                                arrival_distribution="uniform",
                                seed=trial
                            )

                            result = run_mlfq(
                                processes=workload,
                                q1=q1,
                                q2=q2,
                                l1=l1,
                                l2=l2,
                                threshold_t=t
                            )
                            trial_results.append(result)
```

```
                avg = average_results(trial_results)

                row = {
                    "Q1": q1,
                    "Q2": q2,
                    "L1": l1,
                    "L2": l2,
                    "T": t,
                    "Throughput": avg["throughput"],
                    "Average_Turnaround_Time": avg["avg_turnaround_time"],
                    "Average_Waiting_Time": avg["avg_waiting_time"],
                    "Average_Response_Time": avg["avg_response_time"],
                    "Completed_Processes": avg["completed_processes"],
                    "Total_Time": avg["total_time"]
                }
                rows.append(row)

                print(
                    f"Tested Q1={q1}, Q2={q2}, L1={l1}, L2={l2}, T={t} | "
                    f"Waiting={row['Average_Waiting_Time']:.2f}, "
                    f"Throughput={row['Throughput']:.4f}"
                )

    with open(output_file, "w", newline="") as file:
        writer = csv.DictWriter(file, fieldnames=rows[0].keys())
        writer.writeheader()
        writer.writerows(rows)

    best_waiting = min(rows, key=lambda r: r["Average_Waiting_Time"])
    best_response = min(rows, key=lambda r: r["Average_Response_Time"])
    best_throughput = max(rows, key=lambda r: r["Throughput"])

    print("\nGrid search completed.")
    print(f"Results saved to {output_file}")

    print("\nBest configuration by waiting time:")
    print(best_waiting)

    print("\nBest configuration by response time:")
    print(best_response)

    print("\nBest configuration by throughput:")
    print(best_throughput)


if __name__ == "__main__":
    main()
```

Code 3. *Queueing Theory Analysis Source Code*

```
import csv


def mm1(lambda_rate, mu_rate):
    rho = lambda_rate / mu_rate

    if rho >= 1:
        return {
            "Model": "M/M/1",
            "Lambda": lambda_rate,
            "Mu": mu_rate,
            "Servers": 1,
            "Utilization": rho,
            "Queue_Length_Lq": None,
            "Waiting_Time_Wq": None,
            "Stable": "No"
        }

    lq = (rho ** 2) / (1 - rho)
    wq = rho / (mu_rate - lambda_rate)

    return {
        "Model": "M/M/1",
        "Lambda": lambda_rate,
        "Mu": mu_rate,
        "Servers": 1,
        "Utilization": rho,
        "Queue_Length_Lq": lq,
        "Waiting_Time_Wq": wq,
        "Stable": "Yes"
    }


def mms(lambda_rate, mu_rate, servers):
    rho = lambda_rate / (servers * mu_rate)

    # Simplified stability-focused M/M/S comparison.
    # The full Erlang-C equation is more complex, but this is enough for the OS project analysis.
    if rho >= 1:
        lq = None
        wq = None
        stable = "No"
    else:
        lq = (rho ** 2) / (1 - rho)
        wq = rho / ((servers * mu_rate) - lambda_rate)
        stable = "Yes"

    return {
        "Model": "M/M/S",
        "Lambda": lambda_rate,
        "Mu": mu_rate,
        "Servers": servers,
        "Utilization": rho,
```

```
        "Queue_Length_Lq": lq,
        "Waiting_Time_Wq": wq,
        "Stable": stable
    }


def main():
    scenarios = [
        mm1(5, 10),
        mm1(8, 10),
        mm1(12, 10),
        mms(12, 10, 2),
        mms(20, 10, 4),
        mms(30, 10, 4),
    ]

    output_file = "queueing_results.csv"

    with open(output_file, "w", newline="") as file:
        writer = csv.DictWriter(file, fieldnames=scenarios[0].keys())
        writer.writeheader()
        writer.writerows(scenarios)

    print("Queueing Theory Results")
    print("-----------------------")
    for row in scenarios:
        print(row)


    print(f"\nResults saved to {output_file}")


if __name__ == "__main__":
    main()
```

Code 4. *Randomness Study Source Code*

```
import csv
from mlfq_simulation import generate_processes, run_mlfq


TRIALS = 100
NUM_PROCESSES = 50
DISTRIBUTIONS = ["uniform", "exponential", "poisson"]


def average_results(results_list):
    keys = results_list[0].keys()
    return {
        key: sum(r[key] for r in results_list) / len(results_list)
        for key in keys
    }


def main():
    rows = []

    # Use one reasonable configuration found from the parameter study.
    q1 = 20
    q2 = 60
    l1 = 2
    l2 = 3
    t = 200

    print("Starting randomness study...")
    print(f"Trials per distribution: {TRIALS}")

    for distribution in DISTRIBUTIONS:
        trial_results = []

        for trial in range(TRIALS):
            workload = generate_processes(
                num_processes=NUM_PROCESSES,
                arrival_distribution=distribution,
                seed=trial
            )

            result = run_mlfq(
                processes=workload,
                q1=q1,
                q2=q2,
                l1=l1,
                l2=l2,
                threshold_t=t
            )

            trial_results.append(result)

        avg = average_results(trial_results)
```

```
        row = {
            "Distribution": distribution,
            "Throughput": avg["throughput"],
            "Average_Turnaround_Time": avg["avg_turnaround_time"],
            "Average_Waiting_Time": avg["avg_waiting_time"],
            "Average_Response_Time": avg["avg_response_time"],
            "Completed_Processes": avg["completed_processes"],
            "Total_Time": avg["total_time"]
        }

        rows.append(row)

        print(
            f"{distribution}: Waiting={row['Average_Waiting_Time']:.2f}, "
            f"Response={row['Average_Response_Time']:.2f}, "
            f"Throughput={row['Throughput']:.4f}"
        )

    output_file = "distribution_results.csv"

    with open(output_file, "w", newline="") as file:
        writer = csv.DictWriter(file, fieldnames=rows[0].keys())
        writer.writeheader()
        writer.writerows(rows)

    print(f"\nResults saved to {output_file}")


if __name__ == "__main__":
    main()
```

Appendix B

Figures 1 – 5 are screenshots for all the scripts executed and can be found inside folder screenshots.

Figure 1. *Execution of the MLFQ Simulation*

```
artemisahasalami@group22-01:~/Paper3/code$ python3 mlfq_simulation.py
MLFQ Simulation Results
----------------------
throughput: 0.0018
avg_turnaround_time: 14130.6400
avg_waiting_time: 13574.2000
avg_response_time: 745.3800
completed_processes: 50
total_time: 27840
```

Figure 2. *Execution of the Grid Search Optimization*

```
artemisahasalami@group22-01:~/Paper3/code$ python3 optimization.py | tee ../results/optimization_output.txt
Starting grid search optimization...
Trials per configuration: 100
Tested Q1=10, Q2=40, L1=1, L2=2, T=100 | Waiting=10062.71, Throughput=0.0020
Tested Q1=10, Q2=40, L1=1, L2=2, T=200 | Waiting=10062.71, Throughput=0.0020
Tested Q1=10, Q2=40, L1=1, L2=2, T=300 | Waiting=13140.86, Throughput=0.0020
Tested Q1=10, Q2=40, L1=1, L2=3, T=100 | Waiting=10835.58, Throughput=0.0020
Tested Q1=10, Q2=40, L1=1, L2=3, T=200 | Waiting=10835.58, Throughput=0.0020
Tested Q1=10, Q2=40, L1=1, L2=3, T=300 | Waiting=13516.65, Throughput=0.0020
Tested Q1=10, Q2=40, L1=1, L2=4, T=100 | Waiting=11531.62, Throughput=0.0020
Tested Q1=10, Q2=40, L1=1, L2=4, T=200 | Waiting=11531.62, Throughput=0.0020
Tested Q1=10, Q2=40, L1=1, L2=4, T=300 | Waiting=13820.00, Throughput=0.0020
Tested Q1=10, Q2=40, L1=2, L2=2, T=100 | Waiting=10260.76, Throughput=0.0020
Tested Q1=10, Q2=40, L1=2, L2=2, T=200 | Waiting=10260.76, Throughput=0.0020
Tested Q1=10, Q2=40, L1=2, L2=2, T=300 | Waiting=13220.00, Throughput=0.0020
Tested Q1=10, Q2=40, L1=2, L2=3, T=100 | Waiting=11024.67, Throughput=0.0020
Tested Q1=10, Q2=40, L1=2, L2=3, T=200 | Waiting=11024.67, Throughput=0.0020
Tested Q1=10, Q2=40, L1=2, L2=3, T=300 | Waiting=13621.89, Throughput=0.0020
Tested Q1=10, Q2=40, L1=2, L2=4, T=100 | Waiting=11701.26, Throughput=0.0020
Tested Q1=10, Q2=40, L1=2, L2=4, T=200 | Waiting=11701.26, Throughput=0.0020
Tested Q1=10, Q2=40, L1=2, L2=4, T=300 | Waiting=13906.82, Throughput=0.0020
Tested Q1=10, Q2=40, L1=3, L2=2, T=100 | Waiting=10439.36, Throughput=0.0020
Tested Q1=10, Q2=40, L1=3, L2=2, T=200 | Waiting=10439.36, Throughput=0.0020
Tested Q1=10, Q2=40, L1=3, L2=2, T=300 | Waiting=13275.93, Throughput=0.0020
Tested Q1=10, Q2=40, L1=3, L2=3, T=100 | Waiting=11190.74, Throughput=0.0020
Tested Q1=10, Q2=40, L1=3, L2=3, T=200 | Waiting=11190.74, Throughput=0.0020
Tested Q1=10, Q2=40, L1=3, L2=3, T=300 | Waiting=13669.57, Throughput=0.0020
Tested Q1=10, Q2=40, L1=3, L2=4, T=100 | Waiting=11851.37, Throughput=0.0020
Tested Q1=10, Q2=40, L1=3, L2=4, T=200 | Waiting=11851.37, Throughput=0.0020
Tested Q1=10, Q2=40, L1=3, L2=4, T=300 | Waiting=14003.51, Throughput=0.0020
```

Figure 3. *Execution of the Queueing Theory Analysis*

```
artemisahasalami@group22-01:~/Paper3/code$ python3 queue_models.py | tee ../results/queueing_output.txt
Queueing Theory Results
-----------------------
{'Model': 'M/M/1', 'Lambda': 5, 'Mu': 10, 'Servers': 1, 'Utilization': 0.5, 'Queue_Length_Lq': 0.5, 'Waiting_Time_Wq': 0
.1, 'Stable': 'Yes'}
{'Model': 'M/M/1', 'Lambda': 8, 'Mu': 10, 'Servers': 1, 'Utilization': 0.8, 'Queue_Length_Lq': 3.2000000000000015, 'Wait
ing_Time_Wq': 0.4, 'Stable': 'Yes'}
{'Model': 'M/M/1', 'Lambda': 12, 'Mu': 10, 'Servers': 1, 'Utilization': 1.2, 'Queue_Length_Lq': None, 'Waiting_Time_Wq':
 None, 'Stable': 'No'}
{'Model': 'M/M/S', 'Lambda': 12, 'Mu': 10, 'Servers': 2, 'Utilization': 0.6, 'Queue_Length_Lq': 0.8999999999999999, 'Wai
ting_Time_Wq': 0.075, 'Stable': 'Yes'}
{'Model': 'M/M/S', 'Lambda': 20, 'Mu': 10, 'Servers': 4, 'Utilization': 0.5, 'Queue_Length_Lq': 0.5, 'Waiting_Time_Wq':
0.025, 'Stable': 'Yes'}
{'Model': 'M/M/S', 'Lambda': 30, 'Mu': 10, 'Servers': 4, 'Utilization': 0.75, 'Queue_Length_Lq': 2.25, 'Waiting_Time_Wq'
: 0.075, 'Stable': 'Yes'}
```

Figure 4. *Execution of the Randomness Study*

```
artemisahasalami@group22-01:~/Paper3/code$ python3 randomness_study.py | tee ../results/randomness_output.txt
Starting randomness study...
Trials per distribution: 100
uniform: Waiting=12304.04, Response=726.37, Throughput=0.0020
exponential: Waiting=12206.40, Response=732.97, Throughput=0.0020
poisson: Waiting=12256.39, Response=782.14, Throughput=0.0020
```

Figure 5. *Execution of the Visualization Module*

```
artemisahasalami@group22-01:~/Paper3/code$ python3 visualization.py | tee ../results/visualization_output.txt
Visualization files generated:
- heatmap_waiting_time.png
- heatmap_throughput.png
- surface_waiting_time.png
- distribution_comparison.png
- queueing_utilization.png
```